

Grupperkontrakt

Gruppekontrakten beskrevet forneden eksisterer for at skabe en fælles forventningsafstemmelse af samarbejdet, forbygge misforståelser og konflikter samt fastlægge hvordan gruppen i løbet af processen vil fordele arbejde og træffe beslutninger.

Navn	Studienr	Email
Lasse Fink Hansen	202306755	202306755@post.au.dk
Rene Sand Schumacher	202305367	202305367@post.au.dk
Simon Bøjgård Nowack	202308256	202308256@post.au.dk

Table 2: Gruppens medlemmer

1. Fælles mål

Beskriv kort hvad gruppen ønsker at opnå.

Eksempel:

Levere et fagligt stærkt bachelorprojekt. Udvikle et fungerende produkt. Arbejde professionelt og struktureret. Sikre at alle medlemmer lærer noget gennem projektet. Skabe et projekt alle medlemmer kan stå inde for ved den afsluttende eksamen.

2. Arbejdstider og fremmøde

Forneden er gruppens forventet arbejdschema i løbet af ugen inkluderet. Dette inkluderer både individuel studiejob, andre universitets kurser samt møder og fælles arbejde for gruppen:

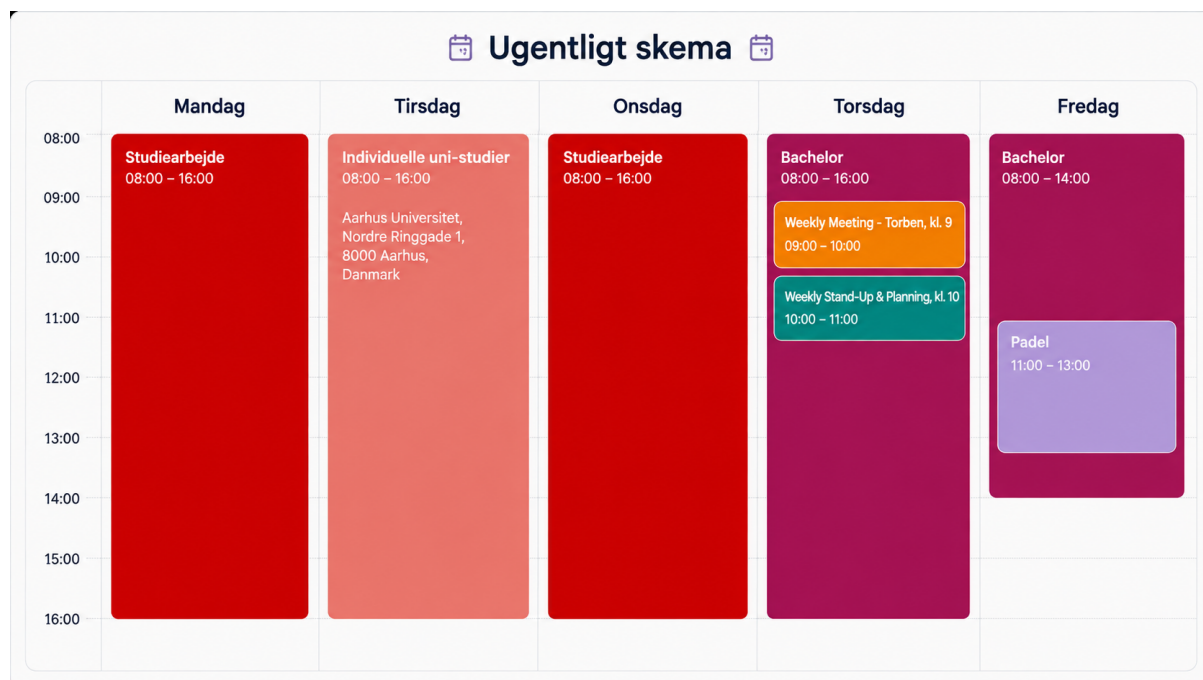


Figure 1: Gruppens ugentlig skema

- Det forventes at arbejde vil foregå ved fysisk fremmøde på hverdage, samt remote arbejde i weekenden, hvis dette er nødvendigt.
- Ved afbud skal det ske senest 24 timer før en aftale, medmindre der er tale om akutte tilfælde (sygdom, tilskadekommelse eller lign.)
- Som udgangspunkt forventes det, at gruppens individuelle medlemmer vil håndtere deres personlige skema (f.eks. afleveringer i andre kurser), så det ikke kommer i vejen for gruppens fastlagte arbejdesgange.

3. Kommunikation

Gruppen vil benytte følgende kommunikationskanaler i forbindelse med projektarbejdet:

- **Discord:** Anvendes til online samarbejde, herunder fælles arbejds-sessioner, skærmdeling og mundtlig kommunikation via voice-kanaler.
- **Messenger:** Anvendes som gruppens primære kanal til hurtig og uformel kommunikation, eksempelvis korte beskeder, koordinering og aftale af mødetidspunkter. Messenger anvendes som udgangspunkt ikke til længere eller komplekse faglige diskussioner.
- **GitHub Issues/Projects:** Anvendes til oprettelse og organisering af tasks samt til fordeling af arbejdsopgaver, opfølgning på fremdrift og generel projektstyring.
- **GitHub Pull Requests:** Anvendes som den primære kvalitetssikringsmekanisme for kode, der ønskes integreret i projektet. Pull Requests bruges til review, diskussion og godkendelse af kodeændringer inden merge til hovedbranchen.
- **GitHub Wiki:** Anvendes til dokumentation af de individuelle repositories samt relevante tekniske beslutninger, opsætninger og arbejds gange i forbindelse med udviklingen.
- **Google Calendar:** Anvendes til koordinering af gruppens fælles og individuelle kalender, herunder faste arbejdsdage, møder, studiearbejde, ferie og øvrigt fravær.

4. Møder

Gruppen afholder som udgangspunkt to faste møder hver torsdag:

- **Vejledermøde:** Møde med gruppens vejleder, Torben. Dagsordenen tilpasses løbende efter projektets aktuelle status og gruppens behov for faglig sparring, feedback og afklaring.
- **Internt stand-up-møde:** Kort møde mellem gruppens medlemmer med en fast dagsorden. Mødet følger en stærk inspireret scrum struktur, hvor hvert medlem gennemgår:
 - Hvad der er blevet arbejdet på siden sidste stand-up.
 - Hvad der arbejdes på frem mod næste stand-up.

- Eventuelle problemer, blokeringer eller forhold, der kræver hjælp eller koordinering.

Lasse er ansvarlig for at tage noter og udarbejde et kort referat fra hvert møde. Mødereferater dokumenteres som Markdown-filer i gruppens **artifacts**-repository, så beslutninger, aftaler og relevante diskussionspunkter løbende dokumenteres og kan genfindes.

Udover de fastlagte møder vil der efter andre scrum-relaterede møder som demo som retrospectives. Disse mødetyper ses ikke som værende nødvendige at have hver uge og vil derfor blive holdt efter behov.

5. Git- og udviklingsregler

For at sikre en stabil, sporbar og ensartet udviklingsproces anvender gruppen fælles regler for versionsstyring, kodekvalitet og review. Reglerne tager udgangspunkt i almindelig praksis inden for professionel softwareudvikling.

Branches

Gruppen anvender en branch-baseret udviklingsmodel, hvor **main**-branchen repræsenterer den seneste stabile version af projektet.

- **main** skal altid være i en fungerende og deploybar tilstand.
- Nye features, fejlrettelser og øvrige ændringer udvikles på separate branches.
- Direkte commits til **main** undgås. Ændringer integreres gennem Pull Requests.
- Branches bør have et afgrænset formål og kun indeholde ændringer relateret til den pågældende opgave.
- Branch-navne følger en fælles navngivningskonvention og skal tydeligt beskrive branchens formål.

Eksempelvis:

- `feature/task-creation`
- `feature/user-profile`
- `fix/login-error`
- `refactor/payment-service`

Hvor det er relevant, kan branch-navnet desuden indeholde et ID fra GitHub Issues eller GitHub Projects, så ændringen nemt kan spores tilbage til den tilhørende opgave.

Pull Requests

Alle ændringer til **main** foretages som udgangspunkt gennem Pull Requests.

- En Pull Request skal beskrive formålet med ændringen og eventuelle væsentlige implementeringsbeslutninger.
- Pull Requests bør være så små og fokuserede som praktisk muligt.
- Minimum ét andet gruppemedlem skal gennemgå og godkende en Pull Request før merge.
- Relevante automatiske checks, eksempelvis build, linting, type checking og tests, skal være bestået før merge.
- Eventuelle review-kommentarer skal være behandlet eller afklaret inden Pull Requesten merges.
- Pull Requests bør kobles til det relevante Issue eller den relevante opgave i GitHub Projects, hvor dette er muligt.

Commits

Commits skal skabe en forståelig og anvendelig historik over projektets udvikling.

- Commits skal være relativt små, fokuserede og repræsentere en logisk ændring.
- Commit messages skal være korte, beskrivende og tydeligt kommunikere formålet med ændringen.
- Ét commit bør så vidt muligt ikke blande flere uafhængige ændringer.
- Commits må ikke indeholde unødvendige filer, genererede filer eller større mængder irrelevant kode.
- Midlertidig debugging-kode, credentials, secrets og lokale konfigurationsfiler må ikke commits.

Commit messages bør følge en ensartet stil. Gruppen kan eksempelvis anvende principperne fra *Conventional Commits*:

- **feat:** add task creation endpoint
- **fix:** handle invalid login credentials
- **refactor:** simplify payment validation
- **test:** add tests for task service

6. Coding Standards

Gruppen følger fælles coding standards for at sikre, at kodebasen er konsistent, læsbar og vedligeholdelsesvenlig på tværs af gruppens medlemmer.

Som overordnet princip prioriteres **simpel, tydelig og forståelig kode frem for unødvendigt komplekse løsninger**. Abstraktioner og generelle løsninger introduceres, når de skaber en reel fordel for læsbarhed, genbrug eller vedligeholdelse.

Følgende principper gælder som udgangspunkt:

- **Naming conventions:** Navne på klasser, metoder, variabler, komponenter og filer følger de gængse conventions for det anvendte programmeringssprog og framework.
- **Formattering:** Automatisk formattering anvendes, hvor det understøttes. Eksempelvis Prettier på frontend eller tilsvarende formatteringsværktøjer for backend.
- **Linting:** Relevante linters anvendes til automatisk at identificere almindelige fejl, inkonsistent kode og potentielle kvalitetsproblemer.
- **Type checking:** Statisk type checking anvendes, hvor teknologien understøtter dette. Typefejl skal som udgangspunkt løses frem for at omgå.
- **Projektstruktur:** Projektets eksisterende arkitektur og mappestruktur skal følges. Nye mapper eller arkitektoniske mønstre introduceres kun, når der er et klart behov.
- **Kommentarer:** Kommentarer anvendes primært til at forklare *hvorfor* en løsning eksisterer, når dette ikke fremgår tydeligt af koden. Kommentarer, der blot beskriver hvad simpel kode gør, bør undgås.
- **TODOs:** TODO-kommentarer må kun anvendes til konkrete og veldefinerede opgaver. Væsentlige TODOs bør desuden registreres som Issues, så de ikke udelukkende eksisterer i kodebasen.
- **Genbrug og abstraktion:** Duplikeret kode bør reduceres, når det giver mening, men abstraktioner bør først introduceres, når de forbedrer løsningen. Gruppen undgår at abstrahere alene med henblik på hypotetiske fremtidige behov.
- **Warnings og fejl:** Nye compiler-, lint- eller type warnings bør som udgangspunkt ikke introduceres.
- **Dependencies:** Nye eksterne dependencies introduceres kun, når de løser et konkret behov og vurderes at give større værdi end kompleksiteten ved at vedligeholde dem.

7. Code Reviews

Code reviews anvendes som en kvalitetskontrol og som en fælles læringsproces. Formålet er både at identificere fejl og at sikre, at ændringer passer ind i projektets arkitektur og fælles udviklingsprincipper.

Revieweren forventes som minimum at vurdere følgende:

- **Funktionalitet:** Implementerer ændringen den ønskede funktionalitet og håndterer relevante edge cases?

- **Læsbarhed:** Er koden forståelig, velstruktureret og passende navngivet?
- **Arkitektur:** Følger ændringen projektets eksisterende arkitektur og designprincipper?
- **Potentielle fejl:** Introducerer ændringen bugs, uønskede sideeffekter, sikkerhedsproblemer eller u hensigtsmæssig fejlhåndtering?
- **Tests:** Er relevante tests implementeret eller opdateret, så de dækker hovedscenarie samt alle edge cases?
- **Coding standards:** Overholder ændringen gruppens aftalte regler for formatering, naming, projektstruktur, type checking og øvrige coding standards?
- **Scope:** Indeholder Pull Requesten kun ændringer, der er relevante for den opgave, den forsøger at løse?

Feedback i reviews skal være konkret, respektfuld og konstruktiv. Kommentarer skal omhandle kode, design og løsning frem for personen, der har skrevet koden.

Hvis en kommentar vedrører en subjektiv præference frem for en nødvendig ændring, bør dette tydeliggøres. Gruppen bør prioritere fejl, vedligeholdelsesproblemer og arkitektoniske konsekvenser frem for mindre personlige præferencer omkring implementeringsstil.

Ved større uenigheder om en løsning bør diskussionen flyttes fra skriftlige review-kommentarer til en direkte samtale mellem gruppens medlemmer, hvorefter den aftalte løsning dokumenteres i Pull Requesten, hvis det er relevant.

8. Manglende deltagelse

For at sikre en fair arbejdsfordeling og et velfungerende samarbejde forventes det, at alle gruppemedlemmer overholder fælles aftaler, kommunikerer åbent og bidrager aktivt til projektet.

Hvis et gruppemedlem gentagne gange ikke lever op til de aftalte forventninger, håndteres det trinvist:

- Ved første tilfælde tages en intern samtale i gruppen med fokus på årsagen til problemet og på at finde en fælles løsning.
- Hvis problemet gentager sig, udarbejdes en tydelig aftale om, hvilke ændringer der forventes, og hvordan situationen skal forbedres.
- Hvis problemet fortsætter trods tidligere aftaler, dokumenteres forløbet, og gruppens vejleder inddrages med henblik på videre håndtering.

Manglende deltagelse kan blandt andet omfatte:

- Gentagne udeblivelser fra aftalte møder eller arbejdsdage uden rimelig varslings.
- Manglende eller utilstrækkelig kommunikation med resten af gruppen.

- Opgaver, der gentagne gange ikke færdiggøres inden for aftalte deadlines.
- En vedvarende og væsentligt ulige arbejdsindsats sammenlignet med resten af gruppen.
- Manglende opfølgning på aftaler, feedback eller tildelte ansvarsområder.

Formålet med denne proces er først og fremmest at identificere og løse problemer internt i gruppen. Eskalering til vejleder anvendes derfor først, når gentagne forsøg på intern afklaring ikke har været tilstrækkelige.

9. Kvalitet vs. tid

Gruppen arbejder ud fra et princip om, at projektets kvalitet skal vurderes i forhold til den tilgængelige tid og projektets faglige formål. Det er derfor vigtigt at undgå, at unødvendig perfektionisme eller mindre væsentlige funktioner forsinker centrale leverancer.

Følgende principper anvendes:

- Aftalte deadlines prioriteres over unødvendig perfektionisme.
- Projektets centrale funktionalitet implementeres først som et *Minimum Viable Product* (MVP), før der bruges tid på ekstra funktionalitet og forbedringer.
- Der lægges særlig vægt på udviklingsprocessen, de tekniske og metodiske valg samt den tilhørende dokumentation, da disse udgør en væsentlig del af bachelorprojektets faglige grundlag.
- *Nice-to-have*-funktionalitet må ikke blokere eller forsinke funktionalitet, der er nødvendig for at opfylde projektets centrale krav.
- Hvis projektets scope viser sig at være større end forventet, kan gruppen i fællesskab beslutte at reducere eller udskyde mindre væsentlig funktionalitet.
- Det prioriteres at levere en mindre, stabil og veldokumenteret løsning frem for en større løsning med utilstrækkelig kvalitet, test eller dokumentation.

Ændringer i projektets scope skal så vidt muligt være bevidste og dokumenterede, således at det fremgår, hvorfor bestemte funktioner er blevet prioriteret, ændret eller fravalgt.

10. Brug af AI-værktøjer

AI-værktøjer må anvendes som støtte i projektarbejdet, så længe brugen sker kritisk, ansvarligt og i overensstemmelse med universitetets gældende regler.

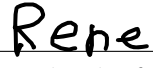
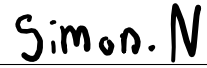
- AI må anvendes til brainstorming, idéudvikling og sparring i forbindelse med både produktudvikling og rapportarbejde.
- AI må anvendes til forslag til kode, arkitektur, testcases og tekniske løsninger.
- AI-genereret kode må ikke anvendes ukritisk. Den ansvarlige udvikler skal forstå, verificere og kunne forklare den kode, der inkluderes i projektet.

- AI må anvendes til sproglig sparring, herunder korrektur, omformulering og forbedring af tekstens struktur og læsbarhed.
- AI-genereret rapporttekst skal altid gennemgås, tilpasses og kvalitetssikres af gruppen. Gruppen er selv ansvarlig for rapportens faglige indhold, argumentation og konklusioner.
- AI-output betragtes som et forslag og ikke som en autoritativ kilde. Faglige påstande og information skal derfor verificeres gennem relevante og troværdige kilder.
- Fortrolige oplysninger, credentials, persondata eller anden følsom information må ikke deles med AI-værktøjer.

Det overordnede princip er, at AI anvendes som et hjælpeværktøj og ikke som en erstatning for gruppens egen faglige vurdering, forståelse og ansvar for det afleverede arbejde.

Underskrifter

Ved underskrift bekræfter gruppens medlemmer, at de har læst og accepteret gruppekontraktens indhold.

Lasse Fink Hansen	 Underskrift	<u>25/08/26</u> Dato
Rene Sand Schumacher	 Underskrift	<u>25/08/26</u> Dato
Simon Bølgård Nowack	 Underskrift	<u>25/08/26</u> Dato